LAPORAN

PRAKTIKUM PEMROGRAMAN PERANGKAT MOBILE

Sesi 01: Pengenalan Flutter Dan Dart



Disusun Oleh:

NAMA : MOCH.ARIEF RIFANSYAH
NIM : 20240040194
KELAS : TI24B

Dosen Pengampu:

YULHAN WAHYUDIN, M.Kom

PROGRAM STUDI TEKNIK INFORMATIKA

UNIVERSITAS NUSA PUTRA

2026

DAFTAR ISI

DAFTAR ISI i
BAB I HASIL VERIFIKASI LINGKUNGAN (ENVIRONMENT SETUP) 1
BAB II STRUKTUR PROYEK & ANALISIS KODE 3
A. Personalisasi & Header (AppBar & Tema) 3
B. Identitas Mahasiswa (Profile Info) 3
C. Kontrol Counter Multi-Fungsi (Logika State & Validasi) 3
D. Perubahan Indikator Visual (Pewarnaan Dinamis) 4
BAB III HASIL UJI COBA APLIKASI 5
A. Pengujian pada Perangkat Virtual (Emulator Android) 5
B. Pengujian pada Perangkat Fisik (Smartphone) 8
BAB IV ANALISIS HOT RELOAD VS HOT RESTART 9
BAB V TROUBLESHOOTING LOG 10

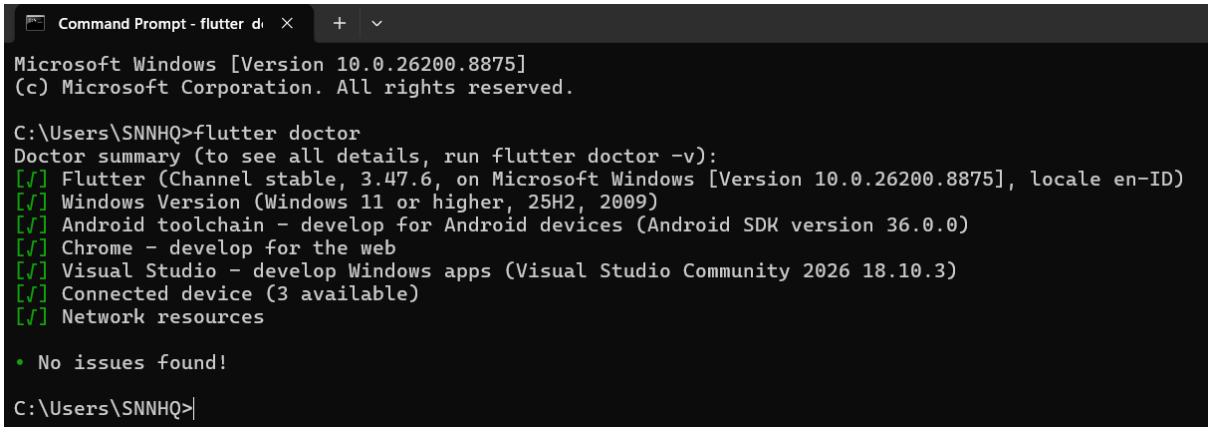
BAB I
HASIL VERIFIKASI LINGKUNGAN (ENVIRONMENT SETUP)

Tahap pertama dalam pengembangan aplikasi cross-platform menggunakan Flutter adalah memastikan seluruh lingkungan pengembangan (development workstation) telah terkonfigurasi dengan benar. Pada praktikum ini, digunakan Visual Studio Code (VS Code) sebagai Integrated Development Environment (IDE) utama, dipadukan dengan Android SDK dan Android Emulator yang dikonfigurasi melalui Android Studio.

Langkah-langkah persiapan yang telah dilakukan meliputi:

1. Mengunduh dan menginstal Flutter SDK versi stabil terbaru, serta mendaftarkan path direktori bin Flutter ke dalam Environment Variables pada sistem operasi Windows.
2. Memasang ekstensi Flutter dan Dart secara resmi di dalam VS Code untuk mendukung proses syntax highlighting, code completion, dan fitur debugging.
3. Membuka SDK Manager pada Android Studio untuk memastikan bahwa Android SDK Platform, Android SDK Command-line Tools, dan Android SDK Build-Tools telah terinstal.
4. Menyetujui seluruh lisensi Android dengan menjalankan perintah flutter doctor --android-licenses di terminal.

Untuk memverifikasi bahwa seluruh *toolchain* sudah siap dipakai tanpa ada dependensi yang tertinggal, dilakukan pengecekan sistem menggunakan perintah `flutter doctor -v` melalui Terminal.



Berdasarkan hasil tangkapan layar di atas, dapat disimpulkan bahwa workstation telah terverifikasi bebas error (ditandai dengan centang hijau) dan siap digunakan untuk proses kompilasi aplikasi ke emulator maupun perangkat fisik.

BAB II
STRUKTUR PROYEK & ANALISIS KODE

Proyek perdana ini diinisiasi menggunakan perintah Command Line Interface (CLI). Struktur proyek dasar mencakup folder android dan ios untuk konfigurasi native, folder lib yang berisi kode sumber utama main.dart, serta file konfigurasi pubspec.yaml.

Pada tugas ini, kode bawaan (Default Counter App) pada file lib/main.dart dimodifikasi secara menyeluruh menggunakan pendekatan StatefulWidget untuk mengelola status data secara reaktif. Berikut adalah analisis mendalam terhadap modifikasi yang diimplementasikan:

A. Personalisasi & Header (AppBar & Tema)

Struktur MaterialApp dikonfigurasi menggunakan tema warna custom agar berbeda dari warna bawaan (default) biru Flutter. Pada bagian Scaffold, widget AppBar dimodifikasi agar menampilkan parameter teks judul yang memuat nama dan NIM: "PPM Sesi 1 - Moch. Arief Rifansyah (20240040194)".

B. Identitas Mahasiswa (Profile Info)

Di dalam struktur layout utama, ditambahkan komponen visual berupa widget Card untuk menampilkan profil mahasiswa. Di dalam Card tersebut, digunakan widget Padding dan Row untuk mengatur tata letak secara horizontal. Di sisi kiri terdapat CircleAvatar dengan ikon pengguna, sedangkan di sisi kanan disusun widget Column berisi teks Nama Lengkap, NIM, dan Kelas dengan pengaturan tipografi tebal (FontWeight.bold).

C. Kontrol Counter Multi-Fungsi (Logika State & Validasi)

Fungsionalitas aplikasi diperluas dengan mendaftarkan tiga method utama di dalam kelas State:

1. **Fungsi Increment:** Menggunakan metode setState() untuk menambahkan variabel *counter* sebesar 1.
2. **Fungsi Decrement (Validasi Batas Minimal):** Diterapkan blok validasi logika (conditional if-else). Jika nilai counter lebih dari 0, maka variabel akan dikurangi 1. Namun, jika nilai sudah mencapai 0, metode setState() ditahan. Sebagai gantinya, sistem memanggil

ScaffoldMessenger.of(context).showSnackBar(...) untuk memunculkan notifikasi pop-up peringatan berwarna merah yang menyatakan: Nilai counter tidak boleh kurang dari 0.

3. **Fungsi Reset:** Mengatur ulang (*reset*) variabel *counter* kembali menjadi 0 di dalam blok setState().

D. Perubahan Indikator Visual (Pewarnaan Dinamis)

Untuk memberikan respons visual berdasarkan angka, diterapkan operasi aritmatika Modulo (_counter % 2 == 0) yang nilainya disimpan dalam variabel boolean isEven.

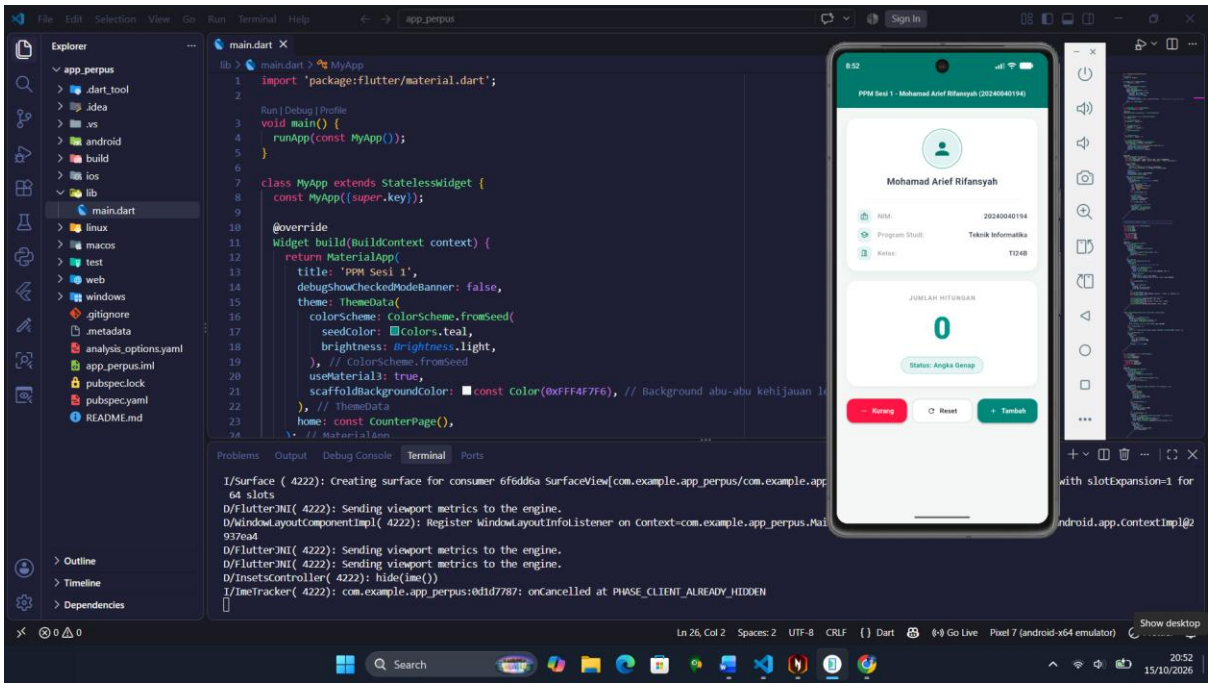
- Jika nilai isEven adalah true (Genap), maka teks angka dan kotak status di-render menggunakan warna Biru dan menampilkan string Status: Angka Genap.
- Jika nilai isEven bernilai false (Ganjil), sistem secara seketika (real-time) menerapkan warna Merah dan merubah teks menjadi Status: Angka Ganjil.

BAB III
HASIL UJI COBA APLIKASI

Untuk memastikan aplikasi berjalan dengan optimal, pengujian (deployment) dilakukan secara komprehensif pada dua target perangkat yang berbeda, yaitu menggunakan Android Virtual Device (Emulator) di PC dan Perangkat Fisik (Real Smartphone Android) melalui USB Debugging.

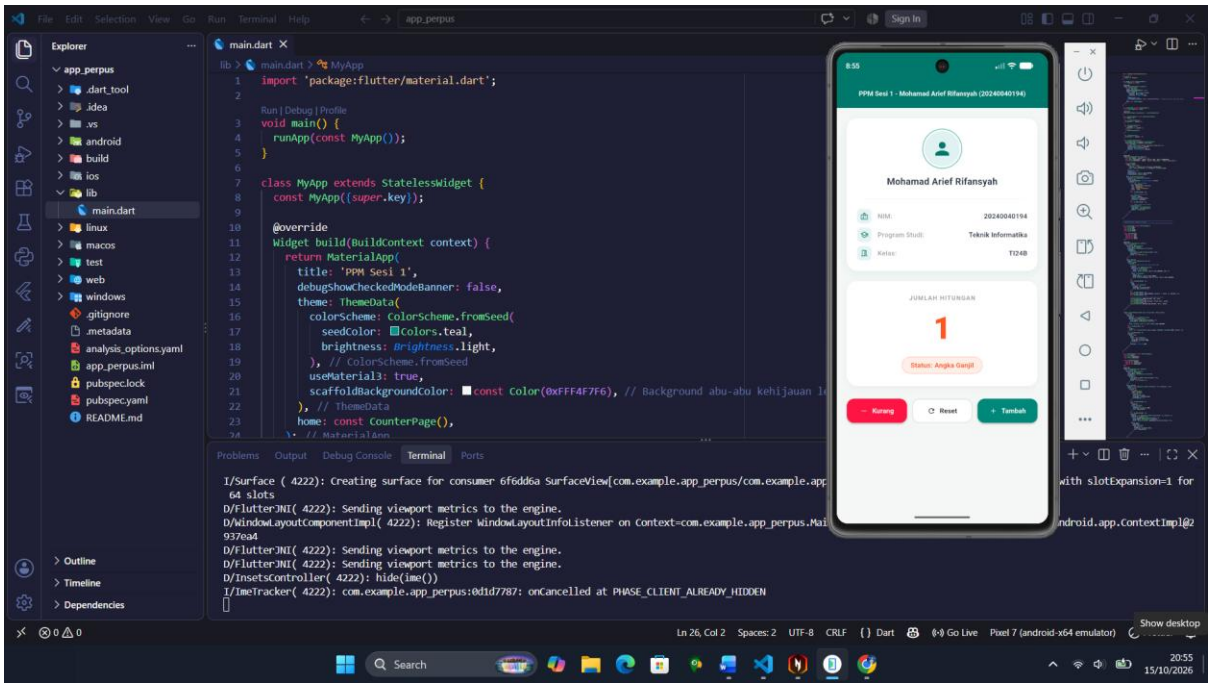
A. Pengujian pada Perangkat Virtual (Emulator Android)

1. Tampilan Awal (State = 0)



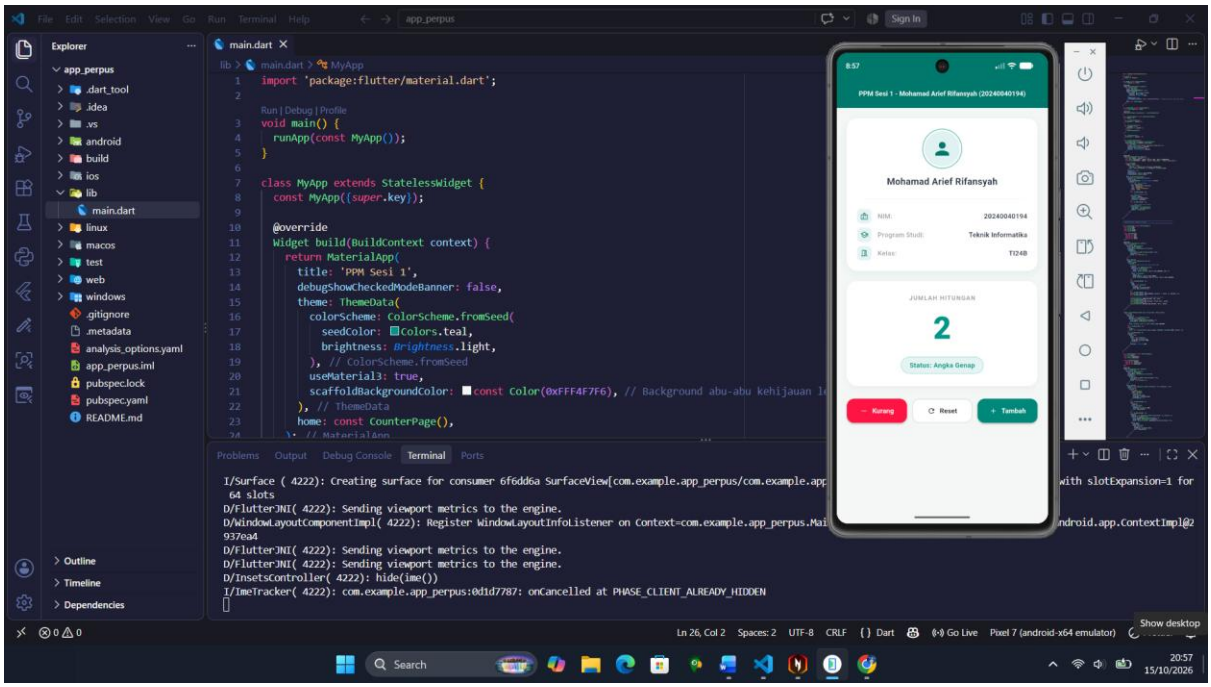
Analisis: Tampilan awal me-render kartu profil mahasiswa di bagian atas. Angka menunjukkan nilai 0 (terdeteksi sebagai genap), sehingga teks berwarna biru dengan label Status: Angka Genap.

2. Saat Counter Bernilai Ganjil



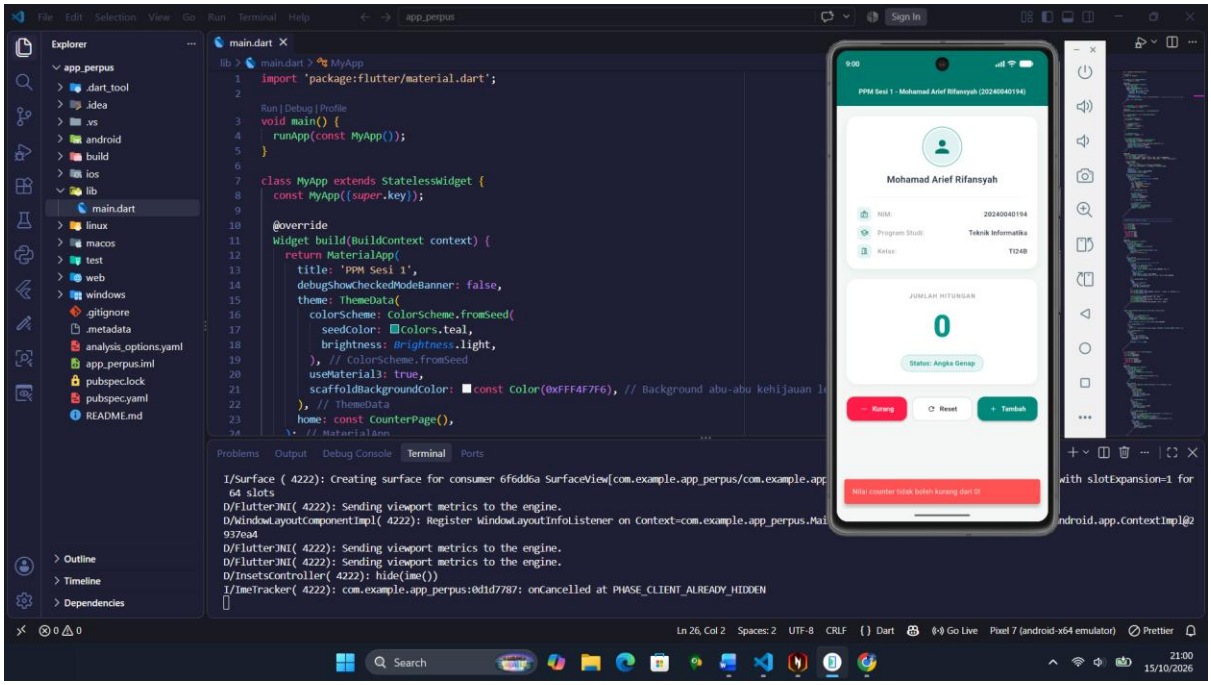
Analisis: Saat ditekan tombol tambah, state berubah menjadi 1 (ganjil). UI merespons perubahan secara otomatis mengubah warna menjadi merah.

3. Saat Counter Bernilai Genap



Analisis: Penambahan nilai menjadi angka genap memicu rebuild UI untuk mengembalikan warna ke mode biru.

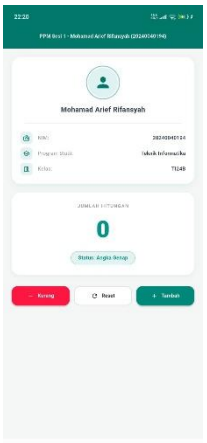
4. Validasi Batas Minimal (Pemicu SnackBar)



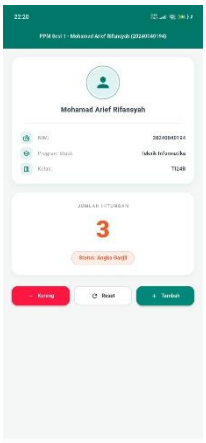
Analisis: Menekan tombol pengurangan pada angka 0 berhasil memicu fungsi validasi dan memunculkan SnackBar batas minimal.

B. Pengujian pada Perangkat Fisik (Smartphone)

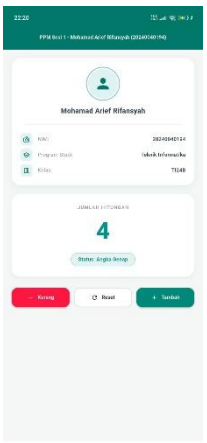
Aplikasi dikompilasi ke format APK dan dijalankan langsung di perangkat fisik. Aplikasi terbukti berjalan mulus tanpa ada *layout* yang terpotong (*overflow*).



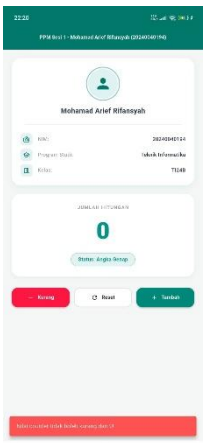
Gambar 1 .1 *Tampilan Awal Aplikasi (Counter = 0)*



Gambar 1 . 2 *Tampilan Saat Counter Bernilai Ganjil*



Gambar 1 .3 *Saat Counter Bernilai Genap*



Gambar 1 .4 *Validasi Batas Minimal*

BAB IV
ANALISIS HOT RELOAD VS HOT RESTART

Berdasarkan praktikum yang dilakukan selama proses pengembangan di VS Code, terdapat perbedaan konseptual dan praktikal yang signifikan antara fitur *Hot Reload* dan *Hot Restart* yang berjalan di atas Dart Virtual Machine (VM):

1. **Hot Reload (Tekan 'r' di Terminal):**

   Memanfaatkan proses Just-In-Time (JIT) compilation milik Dart. Perubahan kode UI langsung disuntikkan ke dalam VM tanpa memuat ulang aplikasi secara penuh, sehingga hanya memakan waktu sangat singkat (kurang dari 1 detik). Karakteristik Utama: Mempertahankan State. Jika angka counter sedang berada di angka 5, lalu dilakukan perubahan style warna kode dan menekan Hot Reload, aplikasi langsung diperbarui dengan UI terbaru, namun angka counter tetap di posisi 5.

2. **Hot Restart (Tekan 'R' di Terminal):**

   Menghentikan eksekusi aplikasi saat ini dan menghancurkan memori State yang sedang berjalan, kemudian memuat dan mengompilasi ulang keseluruhan aplikasi ke kondisi segar (fresh start). Waktu yang dibutuhkan sedikit lebih lama. Karakteristik Utama: Mereset State. Jika angka counter berada di posisi 5, dan dilakukan Hot Restart, aplikasi akan dimuat ulang dari awal dan angka counter akan kembali tereset ke posisi awal (_counter = 0). Fitur ini sangat penting saat ada perubahan fungsi inisialisasi, perubahan file aset, atau struktur utama aplikasi.

BAB V
TROUBLESHOOTING LOG

Selama proses persiapan lingkungan dan kompilasi (build), terdapat beberapa kendala teknis krusial yang berhubungan dengan Android toolchain dan batasan resource sistem:

1. **Kendala: Peringatan Kapasitas Memori (RAM) Emulator**
    - **Gejala:** Saat emulator dijalankan dari VS Code maupun Android Studio, muncul Compatibility Warning yang menyatakan bahwa sistem menyarankan ketersediaan RAM minimal 16 GB. Mengingat laptop hanya memiliki sisa RAM sekitar ~7.9 GB, peringatan ini memicu risiko sistem lag dan kelebihan beban memori.
    - **Solusi:** Optimasi konfigurasi AVD dilakukan melalui menu Device Manager di Android Studio. Pada opsi Edit > Advanced Settings > Memory and Storage, alokasi Virtual RAM emulator diturunkan dari batas default-nya menjadi **2048 MB (2 GB)**. Setelah disimpan (Finish), emulator berjalan jauh lebih responsif, ringan, dan tidak lagi memberatkan laptop saat melakukan coding di VS Code secara bersamaan.
2. **Kendala: Konfigurasi Ruang Tampilan (MainAxisAlignment)**
    - **Gejala:** Saat menyusun antarmuka, sempat terjadi error pada layout baris (Row) akibat kesalahan sintaks pemanggilan konstanta posisi. Muncul peringatan There's no constant named 'between' in 'MainAxisAlignment' yang menghentikan proses build.
    - **Solusi:** Setelah melakukan pengecekan terhadap dokumentasi *widget* di Flutter, dilakukan koreksi kode (revisi sintaks) dari yang sebelumnya tertulis MainAxisAlignment.between menjadi MainAxisAlignment.spaceBetween. Kode berhasil dikompilasi tanpa hambatan setelahnya.